

08 配色

2026/4/13

写在前面：这节课是配色！感觉不用看！（虽然看起来挺赏心悦目的！）😌

本课课纲

- 课前UP主提到常用的颜色索引方法还是颜色名/十六进制颜色。
- 本节课主要讲 R 内置颜色、调色板、自定义颜色与配色方案。

R内置颜色；调色板；自定义颜色；配色方案



R内置颜色

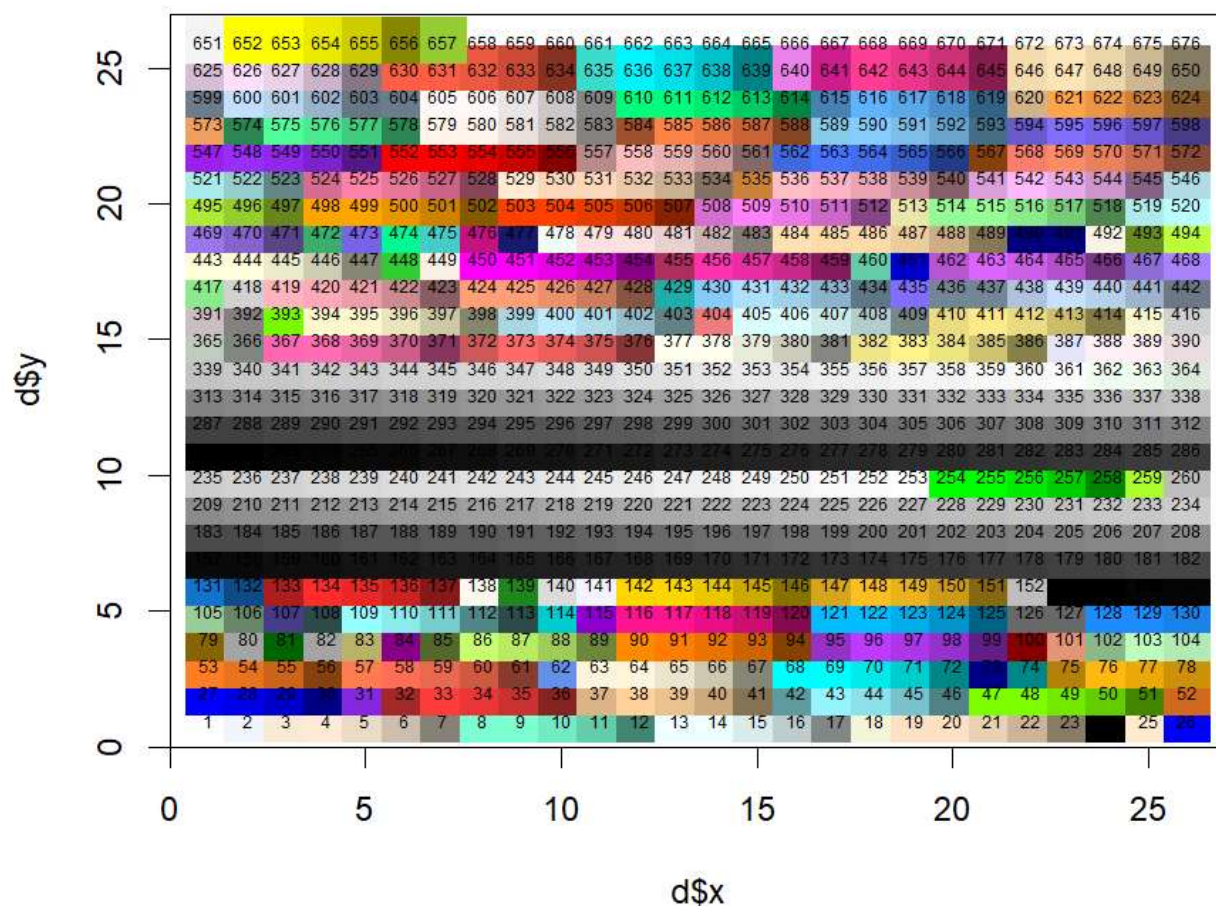
- 使用 `colors()` 函数来查看 R 中内置的颜色。
- 在讲到内置颜色可视化时，用到了 `merge()` 函数，它的作用是按共同列（如日期、ID）把两个数据框合并成一个。在这里，UP主用了这个函数的一个特殊用法：当 `merge()` 合并两个纯数字 / 向量时，它会生成 笛卡尔积（所有两两组合）

```
a <- 1:26
b <- 1:26
d <- merge(a, b) # 一个 26x26 = 676 行的数据框
```

- UP主讲了一个嵌套循环，代码如下：

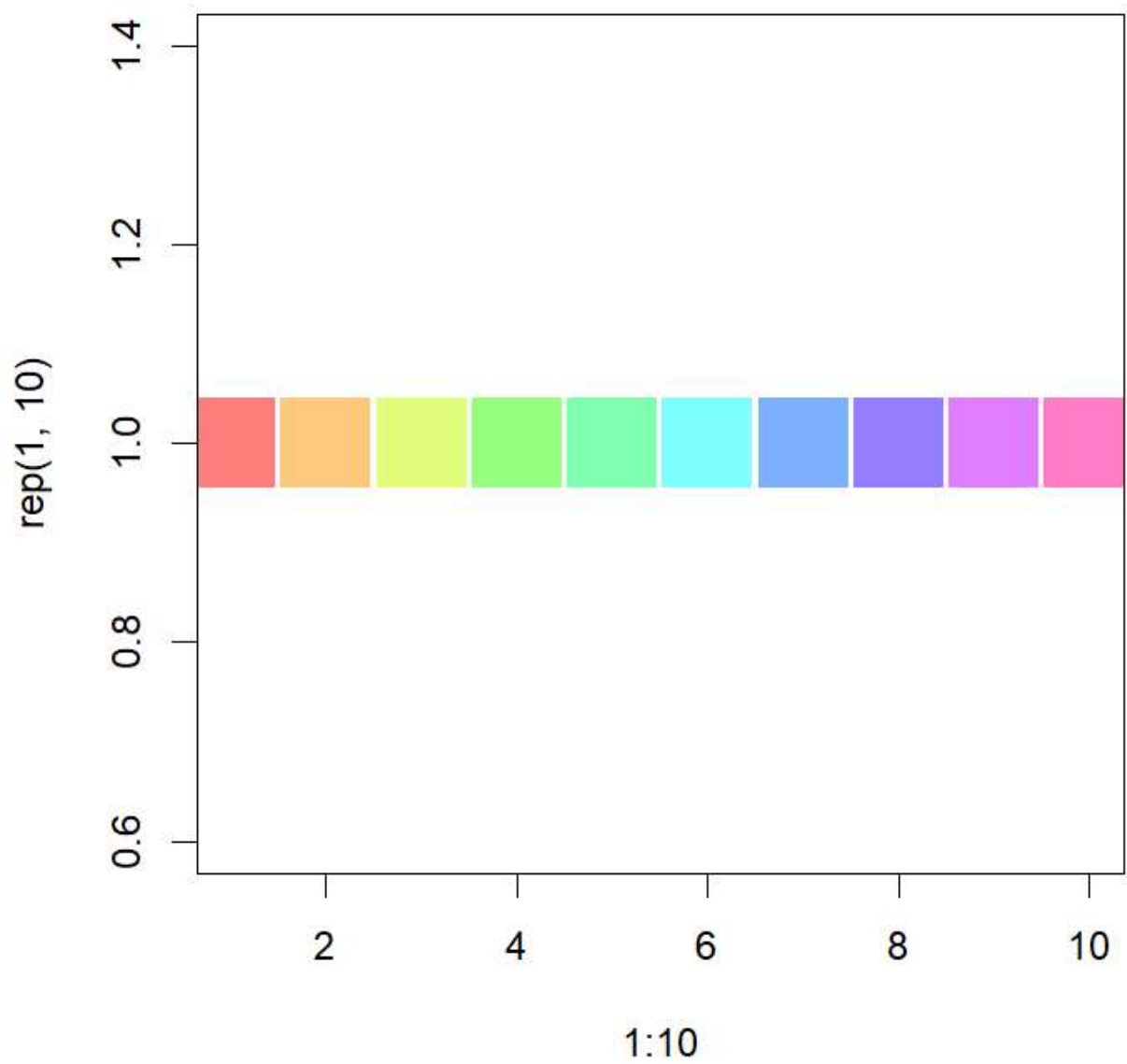
```
plot(d$x, d$y, col=colors(), cex=3, pch=15)
for (i in 1:26) {
  for (j in 1:26) {
    text(i, j, labels=i+(j-1)*26, cex=0.5)
  }
}
```

- 它的含义是，按行、按列双重遍历，在每个点的位置上计算出它对应的编号，并显示出来。
- 画图效果如下：

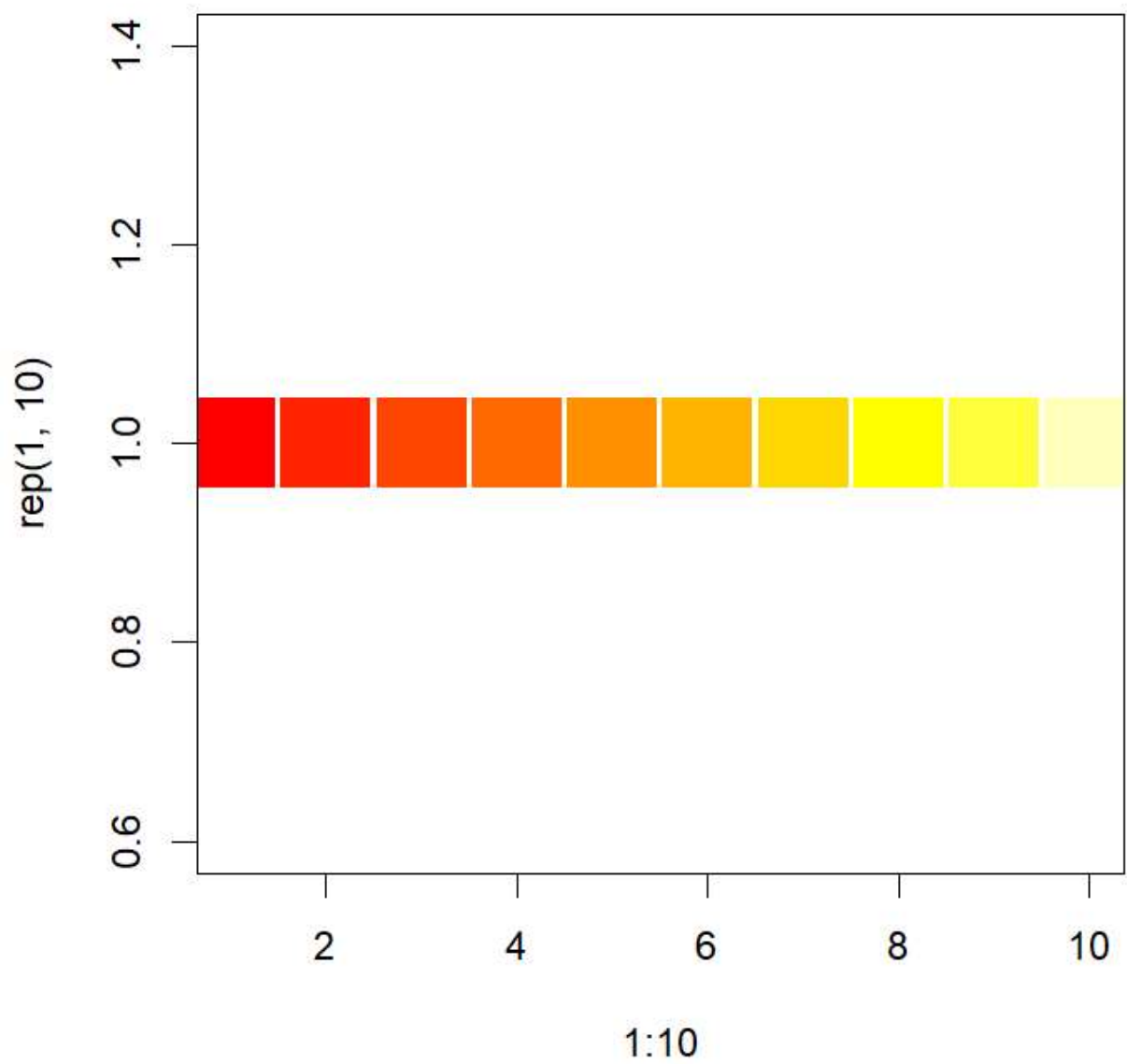


调色板

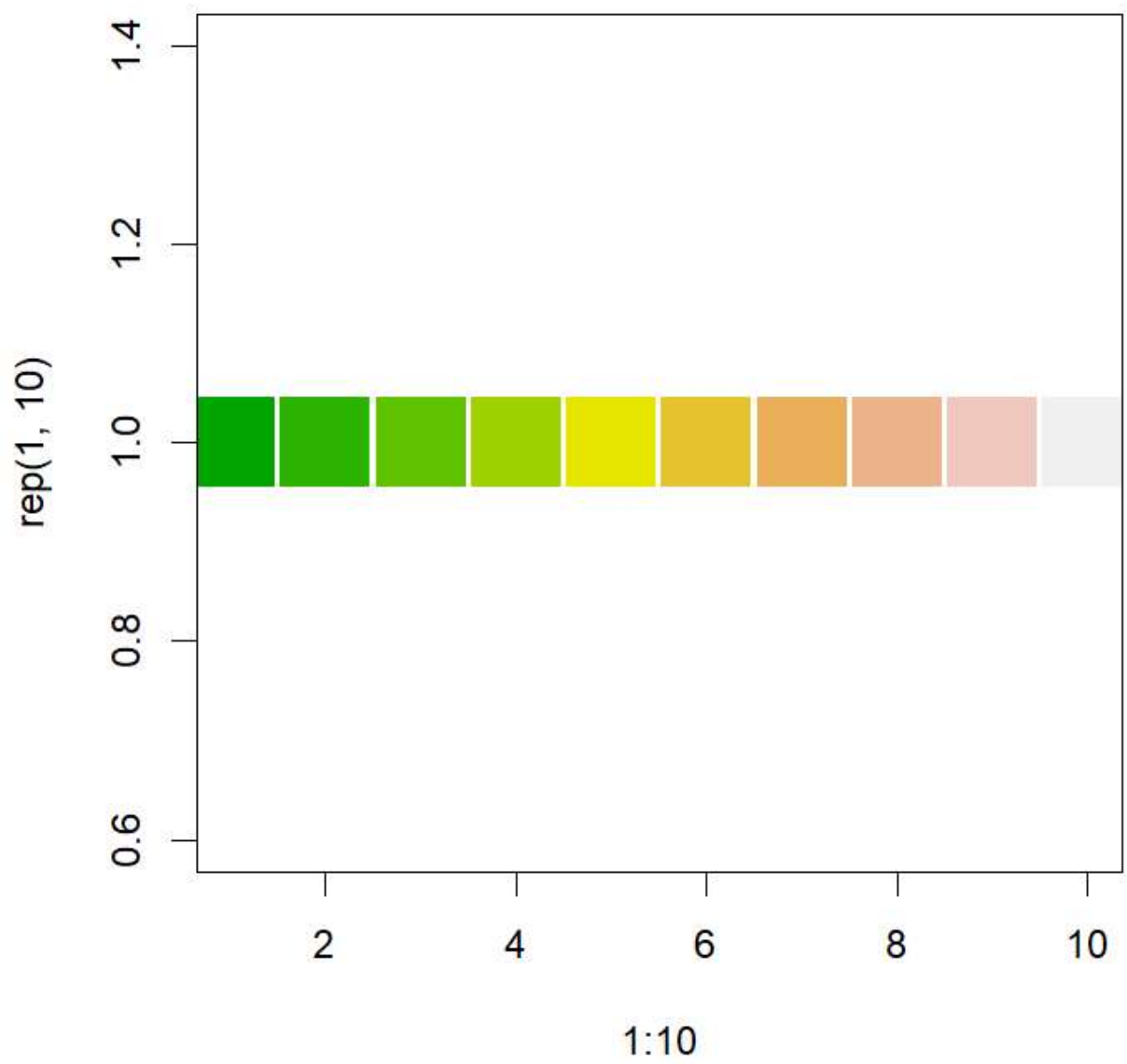
- 调色板是 R 里用来批量生成一组颜色的工具，画图时自动给点 / 线 / 柱子配色。
- 介绍了 R 中内置的若干调色板函数：
 - `rainbow(n)`：彩虹色



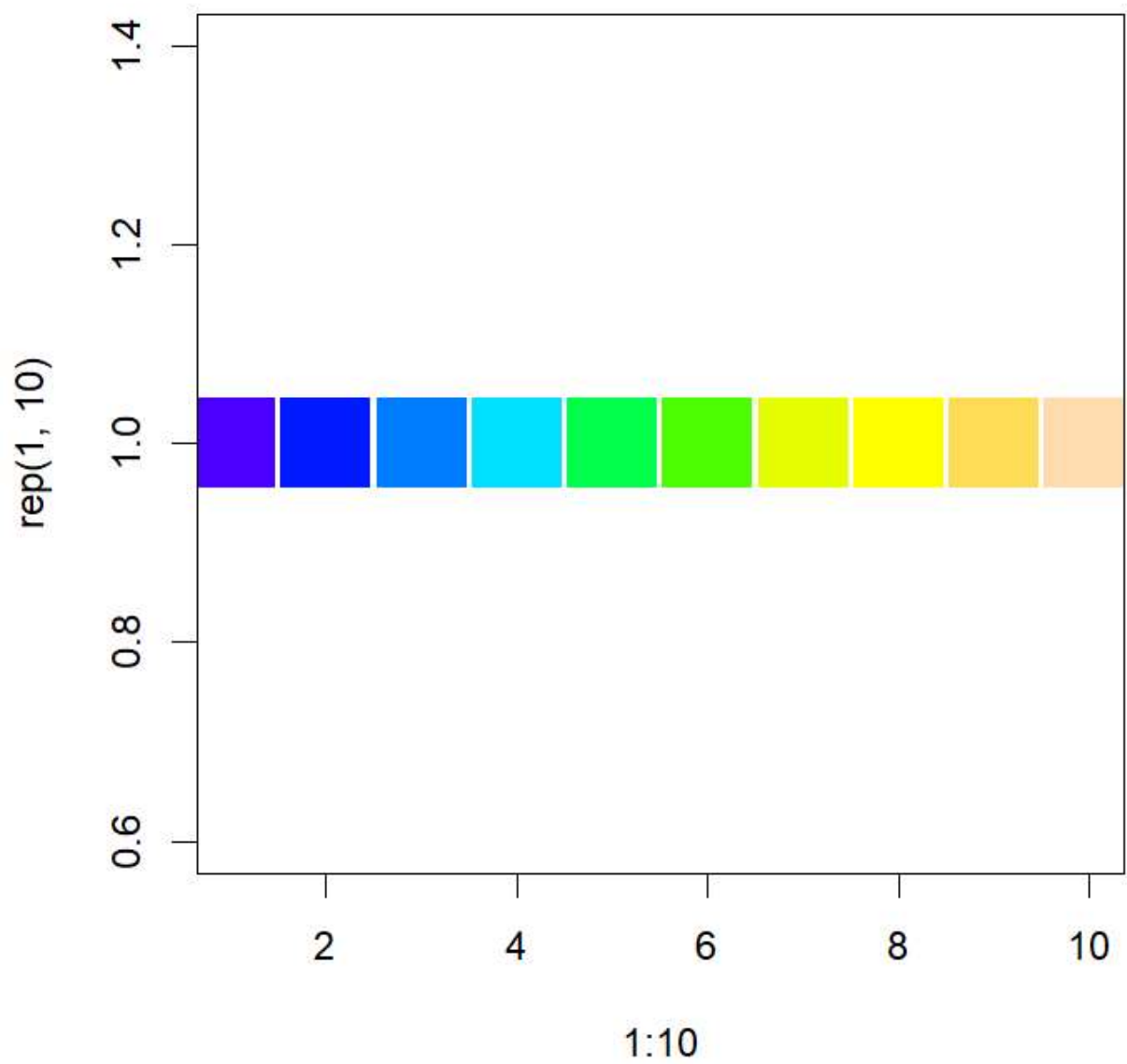
- `heat.colors(n)` : 热色 (红→黄)



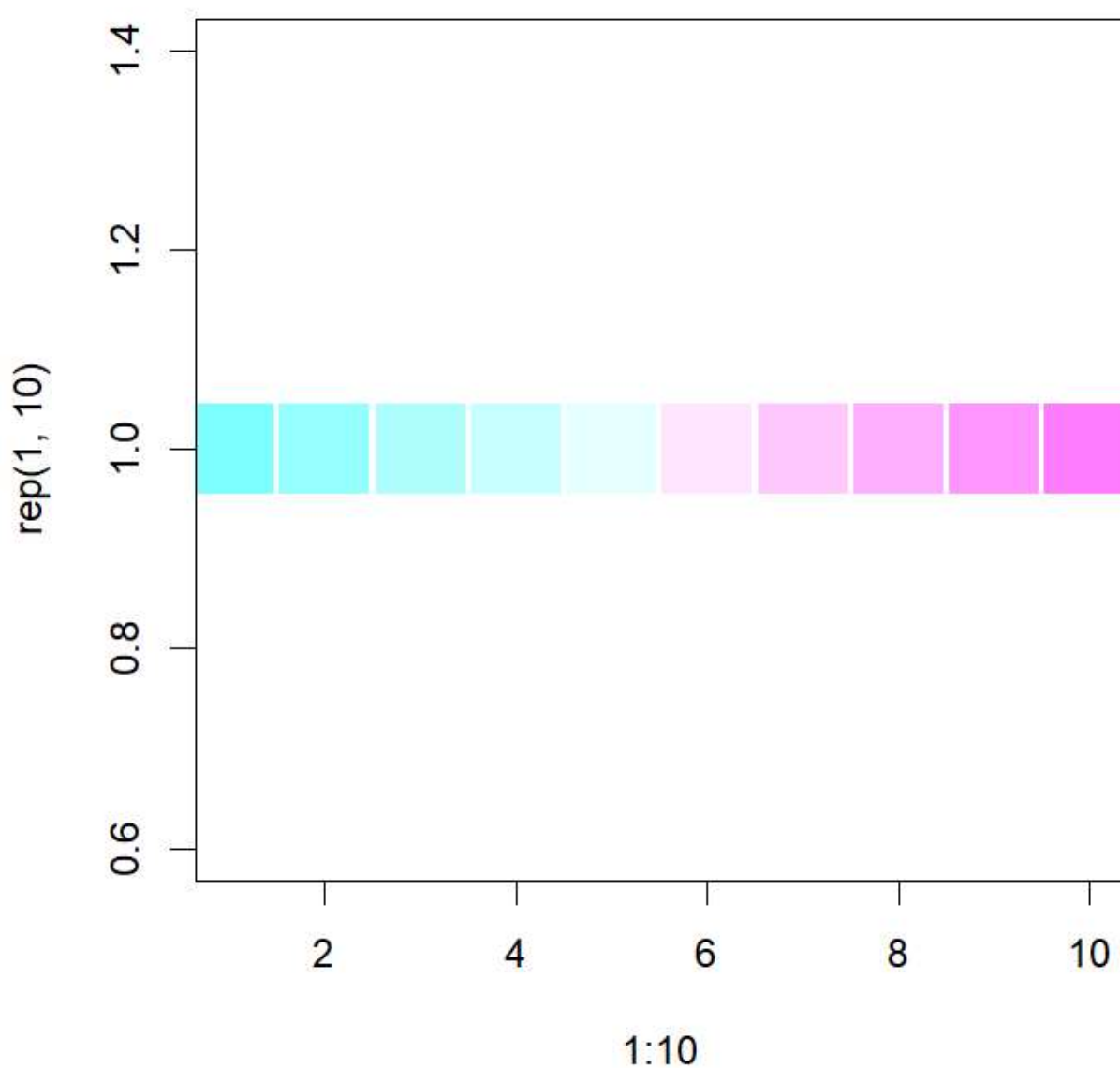
- `terrain.colors(n)` : 地形色



- `topo.colors(n)` : 地形图色



- `cm.colors(n)` : 青→粉



扩展颜色

- 使用 `colorRampPalette()` 函数做**颜色插值**。它的作用是把几种颜色，扩展成任意数量的平滑渐变色。你给它**起始色 + 中间色 + 结束色**，它自动帮你生成一整条顺滑的颜色过渡。
- 这个函数本身的返回结果是一个函数，所以需要在调用的时候传入两次参数，例如：

```
colorRampPalette(c('yellow', 'red'))(50)
```

- 用它生成的绘图示例如下：

```

35 col1 <- colorRampPalette(c('yellow', 'red'))(50)
36 plot(1:50, col=col1, pch=16, cex=2)
37
38
38.1 (Top Level)

```

```

Console Background Jobs
R - R 4.5.3 - D:/Codez/R_Learning/03-系列教学2(BV1DoSjzIEgn)/R201/
> plot(1:10, rep(1, 10), col=rainbow(10, alpha=0.5), pch=15, cex=5)
> ## 热图调色板

```

```

> plot(1:10, rep(1, 10), col=heat.colors(10), pch=15, cex=5)
> ## 地形图调色板
> plot(1:10, rep(1, 10), col=terrain.colors(10), pch=15, cex=5)
> ## topo调色板
> plot(1:10, rep(1, 10), col=topo.colors(10), pch=15, cex=5)
> ## cm调色板
> plot(1:10, rep(1, 10), col=cm.colors(10), pch=15, cex=5)
> ## 扩展颜色

```

```

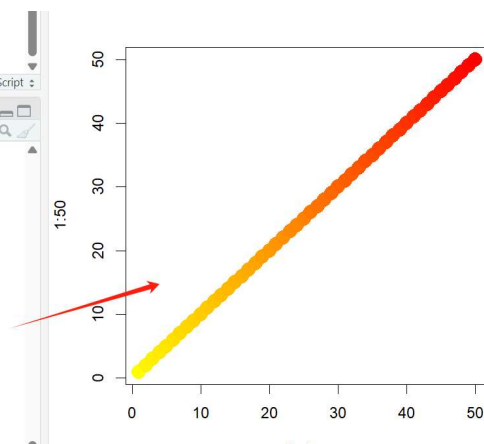
> colorRampPalette(c('yellow', 'red'))(50)
[1] "#FFFF00" "#FFF900" "#FFF400" "#FFE000" "#FFC000" "#FF8000" "#FF4000" "#FF0000"
[10] "#FF0000" "#FF0000" "#FF0000" "#FF0000" "#FF0000" "#FF0000" "#FF0000" "#FF0000"
[19] "#FF0000" "#FF0000" "#FF0000" "#FF0000" "#FF0000" "#FF0000" "#FF0000" "#FF0000"
[28] "#FF0000" "#FF0000" "#FF0000" "#FF0000" "#FF0000" "#FF0000" "#FF0000" "#FF0000"
[37] "#FF0000" "#FF0000" "#FF0000" "#FF0000" "#FF0000" "#FF0000" "#FF0000" "#FF0000"
[46] "#FF0000" "#FF0000" "#FF0000" "#FF0000" "#FF0000" "#FF0000" "#FF0000" "#FF0000"

```

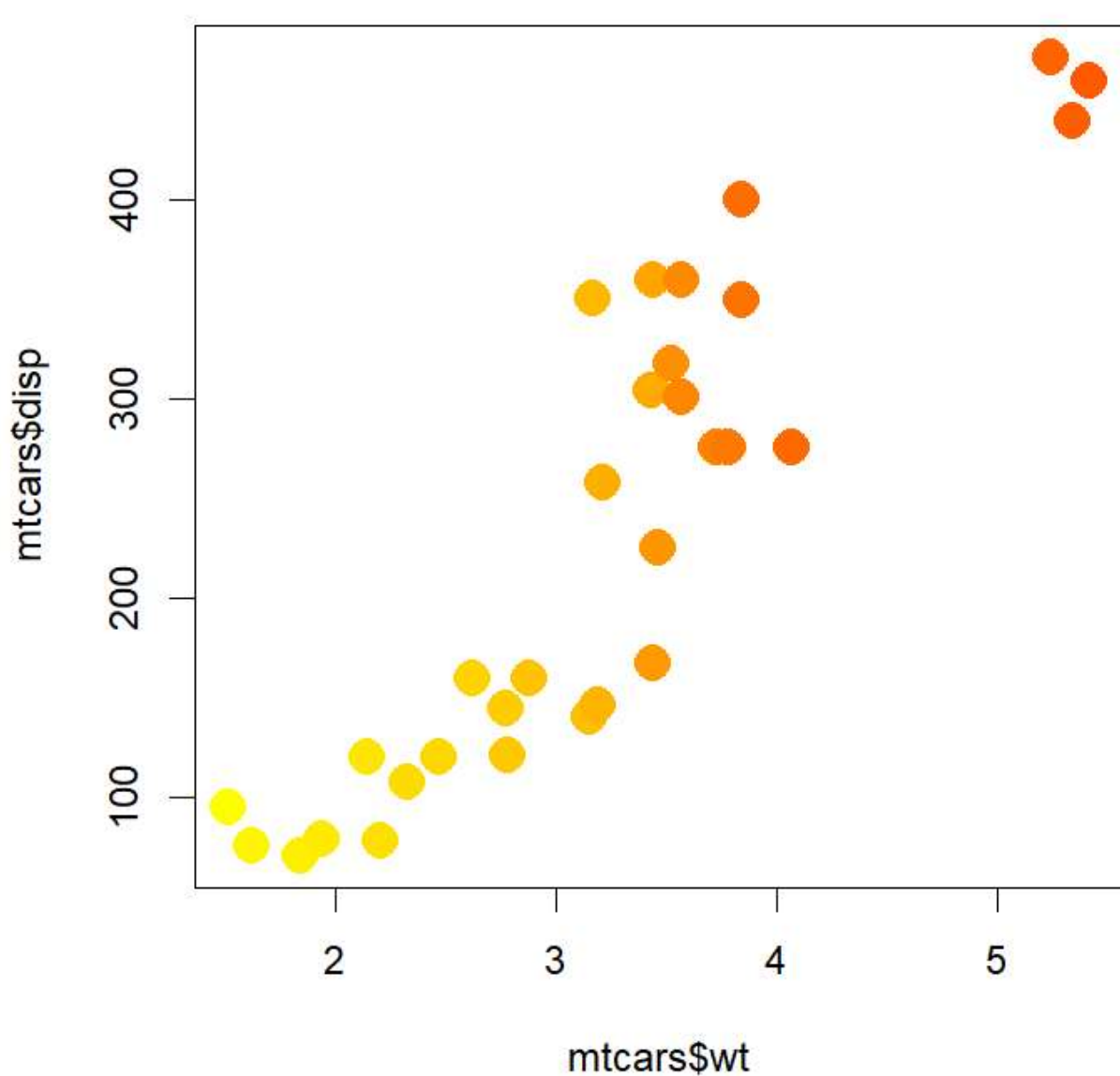
```

> col1 <- colorRampPalette(c('yellow', 'red'))(50)
> plot(1:50, col=col1, pch=16, cex=2)

```

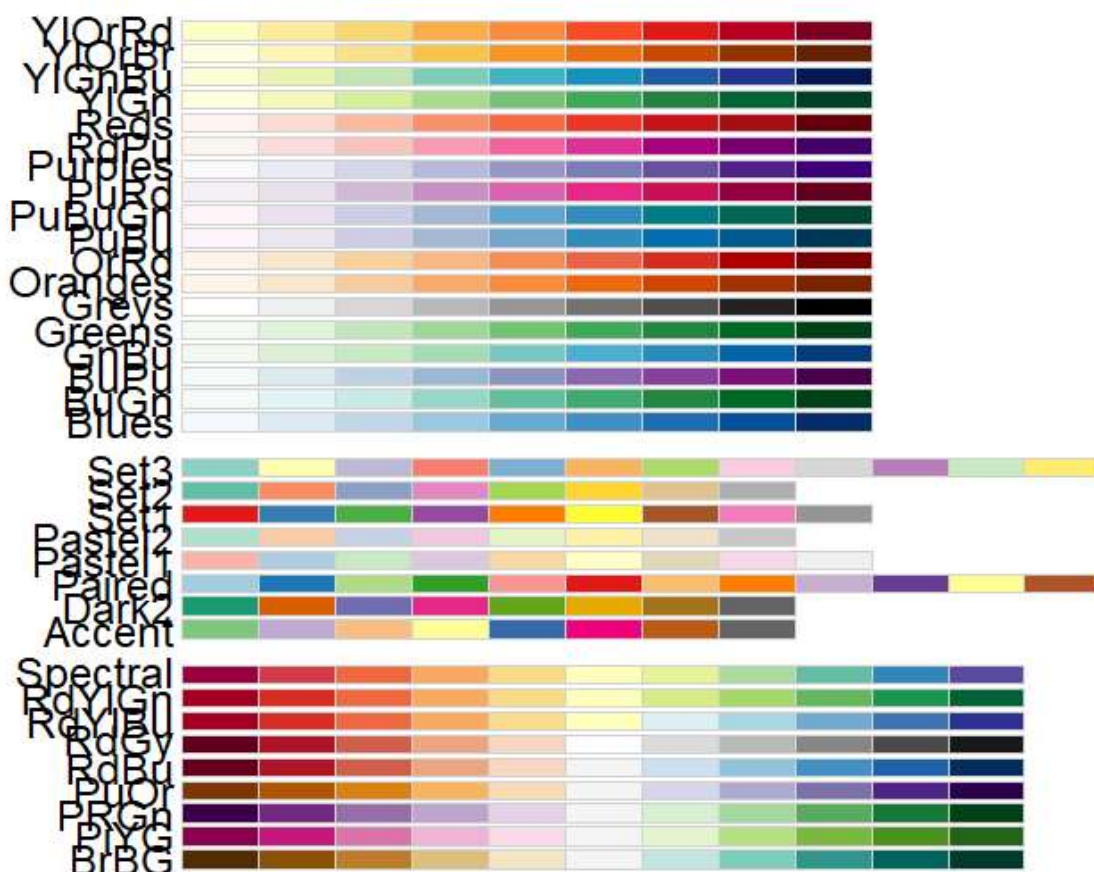


- 也可以用mtcars数据，先按照wt排序，这样颜色就有一定语义了（颜色深浅表示wt的大小）：



配色方案

- 介绍了 RColorBrewer 这个包。
 - 介绍了 `display.brewer.all()` 这个函数，它的作用是展示 RColorBrewer 包中内置的配色方案：



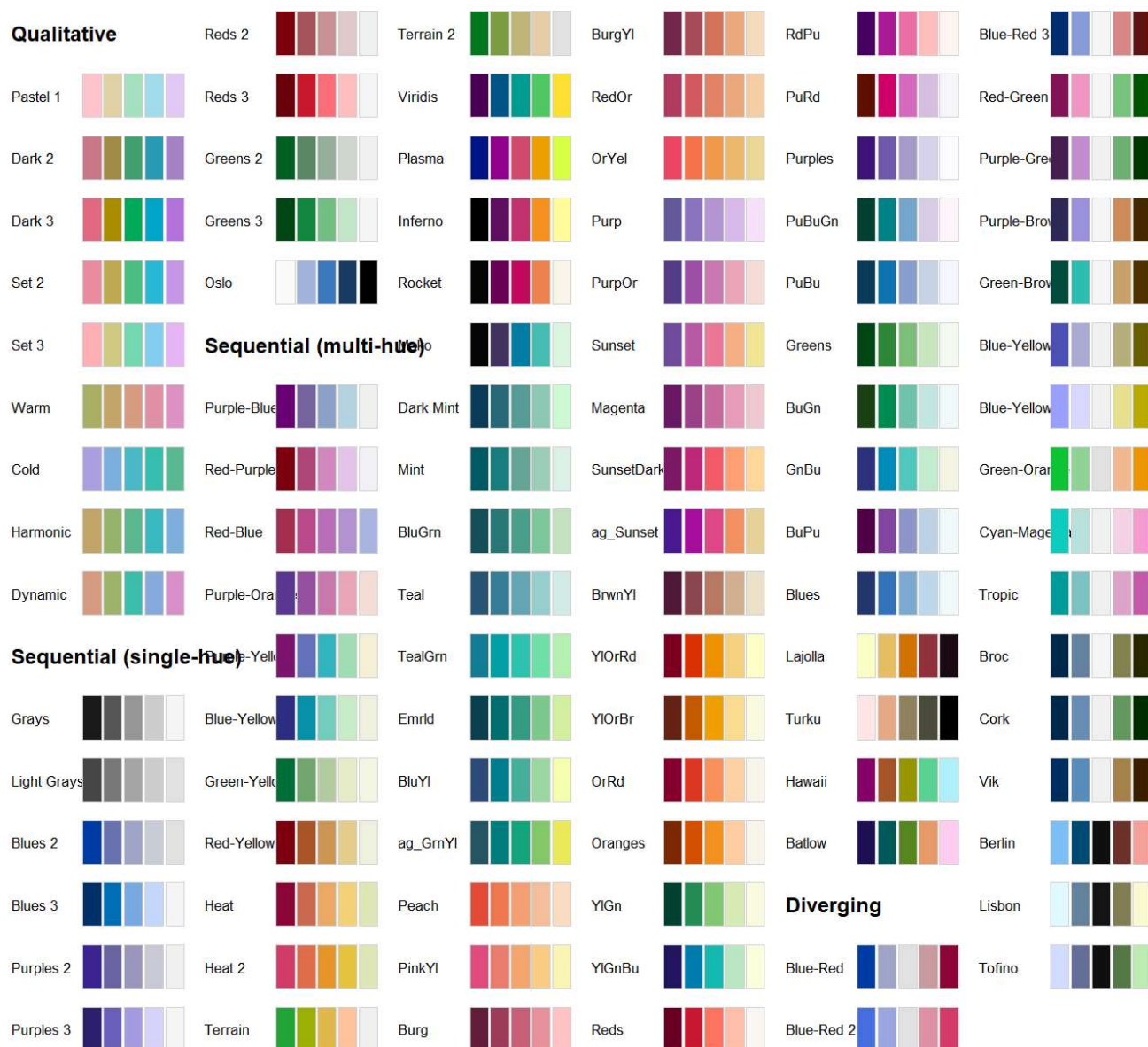
图中有三部分，第一块是渐变的颜色，可以给连续型变量使用；第二块是比较独立的颜色，差异明显，可以用来标记分类型变量；第三块是对比比较强烈的颜色，可以用来标记数据中的离群点。

- 使用 `brewer.pal()` 函数从已有的配色方案中选择颜色：

```
brewer.pal(3, 'Accent') # 从Accent配色方案中选择3种颜色。
```

- 介绍了另一个配色方案的包， `colorspace` ，这个调色板是基于HSV色彩空间的。

- 使用 `hcl_palettes(plot = T)` 来查看配色方案：



包括差异比较明显的（适用于分类变量）的配色方案、单色连续型配色方案、多色连续型配色方案、对比色等。

- 使用调色板里的配色方案，例如，`qualitative_hcl(8, 'warm')`（从 `colorspace -> qualitative -> warm` 系列中选择8个颜色）。